

Курсовая работа

Задача 1.2 — расчёт витрин оборотов и остатков по лицевым счетам в схеме DM

Студент: Орисенко Максим

Дата: 09.05.2026

1. Постановка задачи

После того как детальный слой DS успешно наполнен исходными данными из шести CSV-файлов в задаче 1.1 — нужно рассчитать на его основе две витрины в схеме DM: витрину оборотов по лицевым счетам (`dm.dm_account_turnover_f`) и витрину остатков по лицевым счетам (`dm.dm_account_balance_f`). Структура и описание витрин зафиксированы в выданном файле «Структура таблиц.docx».

По требованию задания нужно реализовать следующее:

- создать процедуру `ds.fill_account_turnover_f(i_OnDate DATE)`, которая по проводкам за дату формирует обороты по каждому счёту, дополнительно пересчитывая суммы в рубли по курсу актуального периода (если курса нет — множитель 1);
- в витрину оборотов попадают только счета, по которым были проводки в эту дату;
- предварительно заполнить витрину остатков `dm.dm_account_balance_f` за 31.12.2017 данными из `ds.ft_balance_f` (один-в-один по полям `on_date/account_rk/balance_out` + расчёт `balance_out_rub` по курсу 31.12.2017);
- создать процедуру `ds.fill_account_balance_f(i_OnDate DATE)`, которая берёт остатки за предыдущий день и применяет к ним обороты текущего дня по правилу: для активных счетов («А») «прошлый остаток + дебет – кредит», для пассивных («П») «прошлый остаток – дебет + кредит»; счёт без оборотов всё равно «протягивается»;
- после реализации — рассчитать обе витрины за каждый день января 2018;
- добавить логирование каждого вызова процедур в таблицу `logs.etl_log` из задачи 1.1 (что считается, время старта/окончания);
- перед расчётом за дату удалять прежние записи за эту же дату, чтобы расчёт можно было перезапускать многократно (идемпотентность).

2. стек и архитектура решения

Используется ровно тот же стек, что и в задаче 1.1: PostgreSQL 13 в Docker (контейнер `hw_coursework_postgres`, порт 5433 проброшен на хост, БД `airflow`, пользователь `airflow`, пароль `airflow`), Python 3 + `psycopg2` для запуска процедур, DBeaver — для проверки результатов и логов. Логика расчёта витрин полностью реализована на стороне PostgreSQL в виде PL/pgSQL-процедур: `ds.fill_account_turnover_f` и `ds.fill_account_balance_f`. Сама БД и схемы `ds`, `logs`, `dm` подняты ранее в рамках 1.1.

Архитектурно витрины DM стоят над детальным слоем DS и пересчитываются ежедневно. Хранилище логирования (logs.etl_log) переиспользуется между всеми ETL-процессами курсовой — отдельной таблицы для 1.2 не создаём, чтобы вся история была в одном месте.

```
PS C:\WINDOWS\System32> docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES          STATUS          PORTS
hw_coursework_postgres Up 12 minutes    0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
PS C:\WINDOWS\System32> |
```

Рис. 1. Контейнер hw_coursework_postgres занят, проброшен порт 5433 → 5432.

В DBeaver видны три рабочие схемы — ds (детальный слой), logs (журнал ETL) и dm (витрины), куда мы и будем писать:

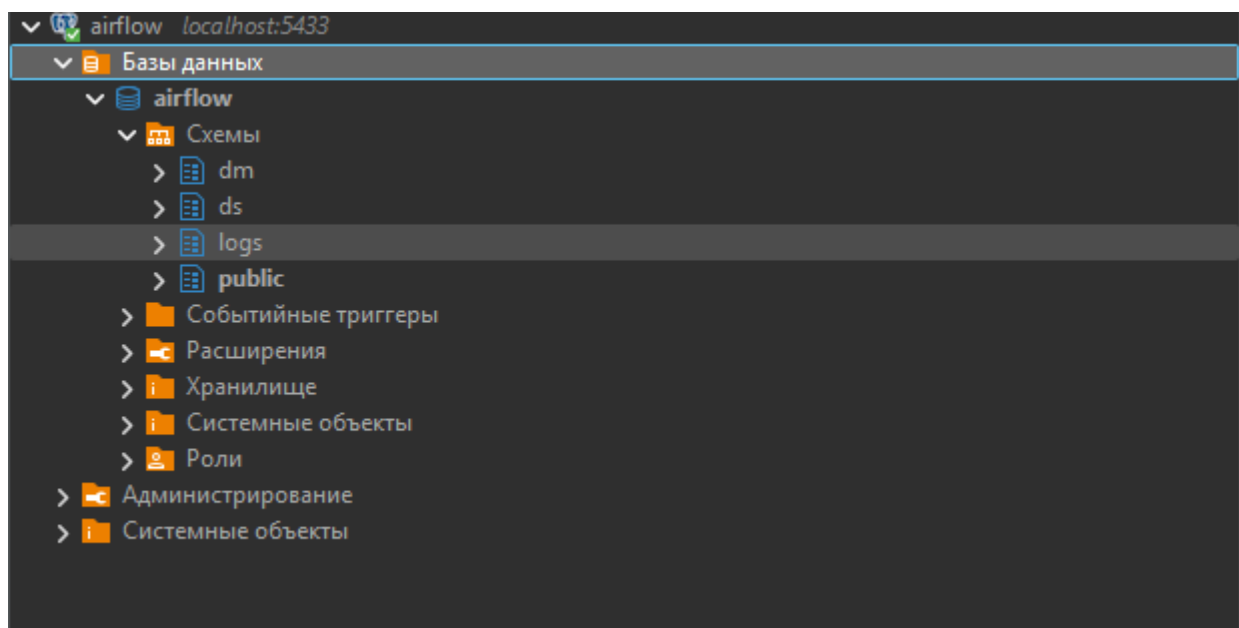


Рис. 2. Дерево схем БД airflow в DBeaver: dm, ds, logs.

Предусловие задачи 1.2 — наполненный DS-слой из задачи 1.1. Контрольные счётчики строк по шести таблицам совпадают с эталонными:

The screenshot shows a SQL script in a text editor with the following query:

```
SELECT 'ft_balance_f' tbl, count(*) FROM ds.ft_balance_f UNION ALL
SELECT 'ft_posting_f', count(*) FROM ds.ft_posting_f UNION ALL
SELECT 'md_account_d', count(*) FROM ds.md_account_d UNION ALL
SELECT 'md_currency_d', count(*) FROM ds.md_currency_d UNION ALL
SELECT 'md_exchange_rate_d', count(*) FROM ds.md_exchange_rate_d UNION ALL
SELECT 'md_ledger_account_s', count(*) FROM ds.md_ledger_account_s;
```

Below the script, the results are displayed in a table view titled "Результат 1". The table has two columns: "tbl" and "count".

tbl	count
ft_balance_f	114
ft_posting_f	33 892
md_account_d	112
md_currency_d	50
md_exchange_rate_d	460
md_ledger_account_s	18

Рис. 3. Сводка по DS: *ft_balance_f*=114, *ft_posting_f*=33 892, *md_account_d*=112, *md_currency_d*=50, *md_exchange_rate_d*=460, *md_ledger_account_s*=18.

3. Структура проекта (фокус на 1.2)

hw_coursework_task1/

├─ data/ 6 CSV-файлов (источник для DS, наполнен 1.1)

├─ sql/

| └─ 01_ddl.sql схемы ds, logs, dm + таблицы

| └─ 02_procedures.sql процедуры fill_account_turnover_f и

| fill_account_balance_f (1.2) + fill_f101_round_f (1.3)

├─ python/

| └─ db.py параметры подключения

| └─ load_csv.py ETL задачи 1.1 (наполнение DS)

| └─ seed_balance_2017_12_31.py предварительное заполнение dm.dm_account_balance_f за 31.12.2017

| └─ run_january.py прогон процедур за каждый день января 2018

| └─ run_f101.py прогон 101 формы (задача 1.3)

└─ talend/ альтернативная сборка ETL под 1.1

4. DDL целевых витрин (sql/01_ddl.sql)

Витрины описаны в той же DDL-спецификации, что и DS-таблицы. Согласно «Структура таблиц.docx», поля числовых оборотов и остатков имеют тип NUMERIC(23,8) — этого достаточно для рублёвого пересчёта с копейками и валютных сумм.

```
82  -- DM: витрины.
83  CREATE TABLE IF NOT EXISTS dm.dm_account_turnover_f (
84      on_date          DATE,
85      account_rk        BIGINT,
86      credit_amount     NUMERIC(23,8),
87      credit_amount_rub NUMERIC(23,8),
88      debet_amount      NUMERIC(23,8),
89      debet_amount_rub  NUMERIC(23,8)
90  );
91
92  CREATE TABLE IF NOT EXISTS dm.dm_account_balance_f (
93      on_date          DATE,
94      account_rk        BIGINT,
95      balance_out       NUMERIC(23,8),
96      balance_out_rub   NUMERIC(23,8)
97  );
98
99  CREATE TABLE IF NOT EXISTS dm.dm_f101_round_f (
100     from_date         DATE,
101     to_date           DATE,
102     chapter           CHAR(1),
103     ledger_account     CHAR(5),
104     characteristic     CHAR(1),
105     balance_in_rub     NUMERIC(23,8),
106     balance_in_val     NUMERIC(23,8),
107     balance_in_total   NUMERIC(23,8),
108     turn_deb_rub       NUMERIC(23,8),
109     turn_deb_val       NUMERIC(23,8),
110     turn_deb_total     NUMERIC(23,8),
111     turn_cre_rub       NUMERIC(23,8),
112     turn_cre_val       NUMERIC(23,8),
113     turn_cre_total     NUMERIC(23,8),
114     balance_out_rub    NUMERIC(23,8),
115     balance_out_val    NUMERIC(23,8),
116     balance_out_total  NUMERIC(23,8)
117 );
```

Рис. 4. Фрагмент 01_ddl.sql со структурой dm.dm_account_turnover_f, dm.dm_account_balance_f и dm.dm_f101_round_f.

Также в DDL присутствует определение dm.dm_f101_round_f — оно не используется в 1.2, но создаётся единым скриптом и понадобится в задаче 1.3 (форма 101).

```
CREATE TABLE IF NOT EXISTS dm.dm_account_turnover_f (
on_date DATE,
```

```

account_rk BIGINT,
credit_amount NUMERIC(23,8),
credit_amount_rub NUMERIC(23,8),
debet_amount NUMERIC(23,8),
debet_amount_rub NUMERIC(23,8)
);

CREATE TABLE IF NOT EXISTS dm.dm_account_balance_f (
on_date DATE,
account_rk BIGINT,
balance_out NUMERIC(23,8),
balance_out_rub NUMERIC(23,8)
);

CREATE TABLE IF NOT EXISTS dm.dm_f101_round_f (
from_date DATE,
to_date DATE,
chapter CHAR(1),
ledger_account CHAR(5),
characteristic CHAR(1),
balance_in_rub NUMERIC(23,8),
balance_in_val NUMERIC(23,8),
balance_in_total NUMERIC(23,8),
turn_deb_rub NUMERIC(23,8),
turn_deb_val NUMERIC(23,8),
turn_deb_total NUMERIC(23,8),
turn_cre_rub NUMERIC(23,8),
turn_cre_val NUMERIC(23,8),
turn_cre_total NUMERIC(23,8),
balance_out_rub NUMERIC(23,8),
balance_out_val NUMERIC(23,8),

```

balance_out_total NUMERIC(23,8)

);

-- Копия для проверки импорта в задании 1.4.

CREATE TABLE IF NOT EXISTS dm.dm_f101_round_f_v2 (LIKE dm.dm_f101_round_f INCLUDING ALL);

5. Процедура витрины оборотов (ds.fill_account_turnover_f)

Логика процедуры:

- логируем событие START в logs.etl_log с тегом on_date= и сохраняем log_id в переменную v_log_id;
- для идемпотентности удаляем все строки за дату расчёта (DELETE WHERE on_date = i_OnDate);
- CTE posting — собираем все проводки за i_OnDate в виде «account_rk + кредит/дебет», объединяя обе стороны через UNION ALL и группируя по счёту;
- CTE rate — для каждого счёта берём курс валюты счёта на i_OnDate из ds.md_exchange_rate_d (LEFT JOIN); если курса нет — COALESCE(rate, 1);
- INSERT в dm.dm_account_turnover_f: для каждого счёта с проводками пишем on_date, account_rk, credit_amount/debet_amount в валюте счёта и те же суммы, помноженные на курс — в рубли;
- фиксируем количество вставленных строк через GET DIAGNOSTICS v_rows = ROW_COUNT;
- логируем событие END с rows_affected, ссылкой parent_log_id на START и длительностью в секундах в поле extra;
- блок EXCEPTION WHEN OTHERS — пишем событие ERROR с текстом SQLERRM и пробрасываем исключение наружу.

В витрину попадают только счета с проводками в i_OnDate (источник — CTE posting), что точно соответствует требованию ТЗ.

```

3  LANGUAGE plpgsql AS $$
4  DECLARE
5      v_log_id BIGINT;
6      v_started TIMESTAMP := clock_timestamp();
7      v_finished TIMESTAMP;
8      v_rows BIGINT;
9  BEGIN
10     -- Идемпотентность: чистим записи за дату, чтобы можно было перезапускать.
11     INSERT INTO logs.etl_log(process, object_name, event, started_at, extra)
12     VALUES ('fill_account_turnover_f', 'dm.dm_account_turnover_f', 'START',
13             v_started, 'on_date=' || i_OnDate)
14     RETURNING log_id INTO v_log_id;
15
16     DELETE FROM dm.dm_account_turnover_f WHERE on_date = i_OnDate;
17
18     WITH posting AS (
19         SELECT account_rk,
20                SUM(credit_amount) AS credit_amount,
21                SUM(debet_amount) AS debet_amount
22         FROM (
23             SELECT credit_account_rk AS account_rk,
24                    credit_amount,
25                    0::numeric AS debet_amount
26             FROM ds.ft_posting_f
27             WHERE oper_date = i_OnDate
28             UNION ALL
29             SELECT debet_account_rk AS account_rk,
30                    0::numeric AS credit_amount,
31                    debet_amount
32             FROM ds.ft_posting_f
33             WHERE oper_date = i_OnDate
34         ) u
35         GROUP BY account_rk
36     ),
37     rate AS (
38         SELECT a.account_rk, COALESCE(er.reduced_cource, 1) AS rate
39         FROM ds.md_account_d a
40         LEFT JOIN ds.md_exchange_rate_d er
41             ON er.currency_rk = a.currency_rk
42            AND i_OnDate BETWEEN er.data_actual_date
43               AND COALESCE(er.data_actual_end_date, DATE '9999-12-31')
44        WHERE i_OnDate BETWEEN a.data_actual_date AND a.data_actual_end_date
45     )
46     INSERT INTO dm.dm_account_turnover_f(on_date, account_rk,
47                                           credit_amount, credit_amount_rub,
48                                           debet_amount, debet_amount_rub)

```

Рис. 5. Процедура fill_account_turnover_f, часть 1: лог START, DELETE и CTE posting + rate.

```

45 )
46 INSERT INTO dm.dm_account_turnover_f(on_date, account_rk,
47                                     credit_amount, credit_amount_rub,
48                                     debit_amount, debit_amount_rub)
49 SELECT i_OnDate,
50        p.account_rk,
51        p.credit_amount,
52        p.credit_amount * COALESCE(r.rate, 1),
53        p.debet_amount,
54        p.debet_amount * COALESCE(r.rate, 1)
55 FROM posting p
56 LEFT JOIN rate r USING (account_rk);
57
58 GET DIAGNOSTICS v_rows = ROW_COUNT;
59 v_finished := clock_timestamp();
60
61 INSERT INTO logs.etl_log(process, object_name, event, started_at, finished_at, rows_affected, extra)
62 VALUES ('fill_account_turnover_f', 'dm.dm_account_turnover_f', 'END',
63         v_started, v_finished, v_rows,
64         format('on_date=%s; parent_log_id=%s; duration_sec=%s',
65              i_OnDate, v_log_id,
66              round(EXTRACT(EPOCH FROM v_finished - v_started)::numeric, 3)));
67 EXCEPTION WHEN OTHERS THEN
68 INSERT INTO logs.etl_log(process, object_name, event, started_at, finished_at, extra)
69 VALUES ('fill_account_turnover_f', 'dm.dm_account_turnover_f', 'ERROR',
70         v_started, clock_timestamp(),
71         format('on_date=%s; parent_log_id=%s; sqlerrm=%s',
72              i_OnDate, v_log_id, SQLERRM));
73 RAISE;
74 END;
75 $$;

```

Рис. 6. Процедура *fill_account_turnover_f*, часть 2: INSERT в *dm*, лог END с *parent_log_id* и *duration_sec*, EXCEPTION.

5.1. Полный текст процедуры

-- Процедуры расчёта витрин.

-- 1.2.a Витрина оборотов.

-- Логика: берём проводки за i_OnDate, складываем в один проход кредитовые и

-- дебетовые обороты по каждому account_rk. Курс — из md_exchange_rate_d, для

-- активного интервала актуальности счёта (md_account_d). Если курса нет, берём 1.

-- В витрину попадают ТОЛЬКО счета, по которым были проводки в этот день.

CREATE OR REPLACE PROCEDURE ds.fill_account_turnover_f(i_OnDate DATE)

LANGUAGE plpgsql AS \$\$

DECLARE

v_log_id BIGINT;

v_started TIMESTAMP := clock_timestamp();

v_finished TIMESTAMP;


```

v_rows BIGINT;

BEGIN

-- Идемпотентность: чистим записи за дату, чтобы можно было перезапускать.

INSERT INTO logs.etl_log(process, object_name, event, started_at, extra)
VALUES ('fill_account_turnover_f', 'dm.dm_account_turnover_f', 'START',
v_started, 'on_date=' || i_OnDate)
RETURNING log_id INTO v_log_id;

DELETE FROM dm.dm_account_turnover_f WHERE on_date = i_OnDate;

WITH posting AS (
SELECT account_rk,
SUM(credit_amount) AS credit_amount,
SUM(debet_amount) AS debet_amount
FROM (
SELECT credit_account_rk AS account_rk,
credit_amount,
0::numeric AS debet_amount
FROM ds.ft_posting_f
WHERE oper_date = i_OnDate
UNION ALL
SELECT debet_account_rk AS account_rk,
0::numeric AS credit_amount,
debet_amount
FROM ds.ft_posting_f
WHERE oper_date = i_OnDate
) u
GROUP BY account_rk
),
rate AS (
SELECT a.account_rk, COALESCE(er.reduced_cource, 1) AS rate

```

```

FROM ds.md_account_d a
LEFT JOIN ds.md_exchange_rate_d er
ON er.currency_rk = a.currency_rk
AND i_OnDate BETWEEN er.data_actual_date
AND COALESCE(er.data_actual_end_date, DATE '9999-12-31')
WHERE i_OnDate BETWEEN a.data_actual_date AND a.data_actual_end_date
)
INSERT INTO dm.dm_account_turnover_f(on_date, account_rk,
credit_amount, credit_amount_rub,
debet_amount, debet_amount_rub)
SELECT i_OnDate,
p.account_rk,
p.credit_amount,
p.credit_amount * COALESCE(r.rate, 1),
p.debet_amount,
p.debet_amount * COALESCE(r.rate, 1)
FROM posting p
LEFT JOIN rate r USING (account_rk);
GET DIAGNOSTICS v_rows = ROW_COUNT;
v_finished := clock_timestamp();
INSERT INTO logs.etl_log(process, object_name, event, started_at, finished_at, rows_affected, extra)
VALUES ('fill_account_turnover_f', 'dm.dm_account_turnover_f', 'END',
v_started, v_finished, v_rows,
format('on_date=%s; parent_log_id=%s; duration_sec=%s',
i_OnDate, v_log_id,
round(EXTRACT(EPOCH FROM v_finished - v_started)::numeric, 3)));
EXCEPTION WHEN OTHERS THEN
INSERT INTO logs.etl_log(process, object_name, event, started_at, finished_at, extra)
VALUES ('fill_account_turnover_f', 'dm.dm_account_turnover_f', 'ERROR',

```

```

v_started, clock_timestamp(),
format('on_date=%s; parent_log_id=%s; sqlerrm=%s',
i_OnDate, v_log_id, SQLERRM));
RAISE;
END;
$$;

```

6. Процедура витрины остатков (ds.fill_account_balance_f)

Логика процедуры:

- логируем событие START в logs.etl_log с on_date=...;
- идемпотентный DELETE строк за i_OnDate;
- CTE active_accounts — все счета, активные в i_OnDate (i_OnDate BETWEEN data_actual_date AND data_actual_end_date);
- CTE prev_bal — остатки за предыдущий день (on_date = i_OnDate - 1 day);
- CTE turn — обороты за i_OnDate из dm.dm_account_turnover_f;
- INSERT с CASE по char_type: «А» → COALESCE(prev,0) + COALESCE(debet,0) - COALESCE(credit,0); «П» → COALESCE(prev,0) - COALESCE(debet,0) + COALESCE(credit,0);
- balance_out_rub считается аналогично, но через рублёвые поля;
- LEFT JOIN с prev_bal и turn + COALESCE 0 — гарантирует, что счёт без оборотов всё равно «протянется» с прошлым остатком;
- лог END с rows_affected, parent_log_id, duration_sec; блок EXCEPTION идентичен оборотам.

```

77 -- 1.2.b Витрина остатков
78 CREATE OR REPLACE PROCEDURE ds.fill_account_balance_f(i_OnDate DATE)
79 LANGUAGE plpgsql AS $$
80 DECLARE
81     v_log_id BIGINT;
82     v_started TIMESTAMP := clock_timestamp();
83     v_finished TIMESTAMP;
84     v_rows BIGINT;
85 BEGIN
86     INSERT INTO logs.etl_log(process, object_name, event, started_at, extra)
87     VALUES ('fill_account_balance_f', 'dm.dm_account_balance_f', 'START',
88             v_started, 'on_date=' || i_OnDate)
89     RETURNING log_id INTO v_log_id;
90
91     DELETE FROM dm.dm_account_balance_f WHERE on_date = i_OnDate;
92
93     WITH active_accounts AS (
94         SELECT account_rk, char_type, currency_rk
95         FROM ds.md_account_d
96         WHERE i_OnDate BETWEEN data_actual_date AND data_actual_end_date
97     ),
98     prev_bal AS (
99         SELECT account_rk, balance_out, balance_out_rub
100        FROM dm.dm_account_balance_f
101        WHERE on_date = i_OnDate - INTERVAL '1 day'
102    ),
103     turn AS (
104         SELECT account_rk, debet_amount, debet_amount_rub,
105                credit_amount, credit_amount_rub
106        FROM dm.dm_account_turnover_f
107        WHERE on_date = i_OnDate
108    )
109     INSERT INTO dm.dm_account_balance_f(on_date, account_rk, balance_out, balance_out_rub)
110     SELECT i_OnDate,
111            a.account_rk,
112            CASE a.char_type
113                WHEN 'A' THEN COALESCE(p.balance_out,0) + COALESCE(t.debet_amount,0) - COALESCE(t.credit_amount,0)
114                WHEN 'П' THEN COALESCE(p.balance_out,0) - COALESCE(t.debet_amount,0) + COALESCE(t.credit_amount,0)
115            END,
116            CASE a.char_type
117                WHEN 'A' THEN COALESCE(p.balance_out_rub,0) + COALESCE(t.debet_amount_rub,0) - COALESCE(t.credit_amount_rub,0)
118                WHEN 'П' THEN COALESCE(p.balance_out_rub,0) - COALESCE(t.debet_amount_rub,0) + COALESCE(t.credit_amount_rub,0)
119            END
120        FROM active_accounts a
121        LEFT JOIN prev_bal p USING (account_rk)
122        LEFT JOIN turn t USING (account_rk);

```

Рис. 7. Процедура `fill_account_balance_f`, часть 1: лог `START`, `DELETE`, CTEs `active_accounts/prev_bal/turn`, `INSERT` с `CASE A/П`.

```

119         END
120     FROM active_accounts a
121     LEFT JOIN prev_bal p USING (account_rk)
122     LEFT JOIN turn    t USING (account_rk);
123
124     GET DIAGNOSTICS v_rows = ROW_COUNT;
125     v_finished := clock_timestamp();
126
127     INSERT INTO logs.etl_log(process, object_name, event, started_at, finished_at, rows_affected, extra)
128     VALUES ('fill_account_balance_f', 'dm.dm_account_balance_f', 'END',
129             v_started, v_finished, v_rows,
130             format('on_date=%s; parent_log_id=%s; duration_sec=%s',
131                   i_OnDate, v_log_id,
132                   round(EXTRACT(EPOCH FROM v_finished - v_started)::numeric, 3)));
133 EXCEPTION WHEN OTHERS THEN
134     INSERT INTO logs.etl_log(process, object_name, event, started_at, finished_at, extra)
135     VALUES ('fill_account_balance_f', 'dm.dm_account_balance_f', 'ERROR',
136             v_started, clock_timestamp(),
137             format('on_date=%s; parent_log_id=%s; sqlerrm=%s',
138                   i_OnDate, v_log_id, SQLERRM));
139     RAISE;
140 END;
141 $$;
142
143 -- 1.3 Расчёт 101 формы.
144 -- i_OnDate — первый день месяца, следующего за отчётным.
145 CREATE OR REPLACE PROCEDURE dm.fill_f101_round_f(i_OnDate DATE)
146 LANGUAGE plpgsql AS $$
147 DECLARE
148     v_started    TIMESTAMP := clock_timestamp();
149     v_from_date  DATE := (i_OnDate - INTERVAL '1 month')::date;
150     v_to_date    DATE := (i_OnDate - INTERVAL '1 day')::date;
151     v_prev_date  DATE := v_from_date - INTERVAL '1 day';
152     v_rows       BIGINT;
153 BEGIN
154     INSERT INTO logs.etl_log(process, object_name, event, started_at, extra)
155     VALUES ('fill_f101_round_f', 'dm.dm_f101_round_f', 'START',
156             v_started, format('period=%s..%s', v_from_date, v_to_date));
157
158     DELETE FROM dm.dm_f101_round_f
159     WHERE from_date = v_from_date AND to_date = v_to_date;
160
161     WITH accounts_period AS (
162         -- счета, действующие в любой день отчётного периода
163         SELECT DISTINCT
164             a.account_rk,
165             substr(a.account_number,1,5) AS ledger_account,

```

Рис. 8. Процедура `fill_account_balance_f`, часть 2 (END/EXCEPTION) и начало `fill_f101_round_f` — соседняя процедура задачи 1.3 в том же файле.

6.1. Полный текст процедуры

- Логика: для всех счетов, действующих на `i_OnDate`, считаем остаток как
- остаток за предыдущий день +/- обороты (`debit_amount/credit_amount`). Знак
- зависит от характеристики счёта: 'A' (активный) +debit–credit,
- 'П' (пассивный) –debit+credit. Если оборотов в этот день не было, остаток
- всё равно «протягивается» с предыдущего дня (LEFT JOIN turn + COALESCE 0).

```

CREATE OR REPLACE PROCEDURE ds.fill_account_balance_f(i_OnDate DATE)

LANGUAGE plpgsql AS $$

DECLARE

v_log_id BIGINT;

v_started TIMESTAMP := clock_timestamp();

v_finished TIMESTAMP;

v_rows BIGINT;

BEGIN

INSERT INTO logs.etl_log(process, object_name, event, started_at, extra)

VALUES ('fill_account_balance_f', 'dm.dm_account_balance_f', 'START',

v_started, 'on_date=' || i_OnDate)

RETURNING log_id INTO v_log_id;

DELETE FROM dm.dm_account_balance_f WHERE on_date = i_OnDate;

WITH active_accounts AS (

SELECT account_rk, char_type, currency_rk

FROM ds.md_account_d

WHERE i_OnDate BETWEEN data_actual_date AND data_actual_end_date

),

prev_bal AS (

SELECT account_rk, balance_out, balance_out_rub

FROM dm.dm_account_balance_f

WHERE on_date = i_OnDate - INTERVAL '1 day'

),

turn AS (

SELECT account_rk, debet_amount, debet_amount_rub,

credit_amount, credit_amount_rub

FROM dm.dm_account_turnover_f

WHERE on_date = i_OnDate

)

```

```

INSERT INTO dm.dm_account_balance_f(on_date, account_rk, balance_out, balance_out_rub)

SELECT i_OnDate,

a.account_rk,

CASE a.char_type

WHEN 'A' THEN COALESCE(p.balance_out,0) + COALESCE(t.debet_amount,0) -
COALESCE(t.credit_amount,0)

WHEN 'IT' THEN COALESCE(p.balance_out,0) - COALESCE(t.debet_amount,0) +
COALESCE(t.credit_amount,0)

END,

CASE a.char_type

WHEN 'A' THEN COALESCE(p.balance_out_rub,0) + COALESCE(t.debet_amount_rub,0) -
COALESCE(t.credit_amount_rub,0)

WHEN 'IT' THEN COALESCE(p.balance_out_rub,0) - COALESCE(t.debet_amount_rub,0) +
COALESCE(t.credit_amount_rub,0)

END

FROM active_accounts a

LEFT JOIN prev_bal p USING (account_rk)

LEFT JOIN turn t USING (account_rk);

GET DIAGNOSTICS v_rows = ROW_COUNT;

v_finished := clock_timestamp();

INSERT INTO logs.etl_log(process, object_name, event, started_at, finished_at, rows_affected, extra)

VALUES ('fill_account_balance_f', 'dm.dm_account_balance_f', 'END',

v_started, v_finished, v_rows,

format('on_date=%s; parent_log_id=%s; duration_sec=%s',

i_OnDate, v_log_id,

round(EXTRACT(EPOCH FROM v_finished - v_started)::numeric, 3)));

EXCEPTION WHEN OTHERS THEN

INSERT INTO logs.etl_log(process, object_name, event, started_at, finished_at, extra)

VALUES ('fill_account_balance_f', 'dm.dm_account_balance_f', 'ERROR',

v_started, clock_timestamp(),

```

```
format('on_date=%s; parent_log_id=%s; sqlerrm=%s',
i_OnDate, v_log_id, SQLERRM));

RAISE;

END;

$$;
```

7. Соседняя процедура fill_f101_round_f (контекст 1.3)

В одном файле sql/02_procedures.sql также лежит процедура расчёта 101 формы для задачи 1.3 — её код виден на тех же скриншотах сразу после fill_account_balance_f. В рамках 1.2 она не используется.

```
160
161 WITH accounts_period AS (
162     -- счета, действующие в любой день отчётного периода
163     SELECT DISTINCT
164         a.account_rk,
165         substr(a.account_number,1,5) AS ledger_account,
166         a.char_type,
167         a.currency_code
168     FROM ds.md_account_d a
169     WHERE a.data_actual_date <= v_to_date
170           AND a.data_actual_end_date >= v_from_date
171 ),
172 chapters AS (
173     SELECT ledger_account::text AS ledger_account, MAX(chapter) AS chapter
174     FROM ds.md_ledger_account_s
175     GROUP BY ledger_account
176 ),
177 bal_in AS (
178     SELECT account_rk, balance_out_rub
179     FROM dm.dm_account_balance_f
180     WHERE on_date = v_prev_date
181 ),
182 bal_out AS (
183     SELECT account_rk, balance_out_rub
184     FROM dm.dm_account_balance_f
185     WHERE on_date = v_to_date
186 ),
187 turn_period AS (
188     SELECT account_rk,
189         SUM(debet_amount_rub) AS turn_deb_rub_total,
190         SUM(credit_amount_rub) AS turn_cre_rub_total
191     FROM dm.dm_account_turnover_f
192     WHERE on_date BETWEEN v_from_date AND v_to_date
193     GROUP BY account_rk
194 ),
195 enriched AS (
196     SELECT a.ledger_account,
197         c.chapter,
198         a.char_type,
199         a.currency_code,
200         COALESCE(bi.balance_out_rub, 0) AS bal_in_rub,
201         COALESCE(bo.balance_out_rub, 0) AS bal_out_rub,
202         COALESCE(tp.turn_deb_rub_total, 0) AS turn_deb_rub_total,
203         COALESCE(tp.turn_cre_rub_total, 0) AS turn_cre_rub_total
```


Рис. 9. *fill_f101_round_f* — CTEs *accounts_period*/*chapters*/*bal_in*/*bal_out*/*turn_period*/*enriched*.

```
199         a.currency_code,  
200         COALESCE(bi.balance_out_rub, 0) AS bal_in_rub,  
201         COALESCE(bo.balance_out_rub, 0) AS bal_out_rub,  
202         COALESCE(tp.turn_deb_rub_total, 0) AS turn_deb_rub_total,  
203         COALESCE(tp.turn_cre_rub_total, 0) AS turn_cre_rub_total  
204     FROM accounts_period a  
205     LEFT JOIN chapters      c ON c.ledger_account = a.ledger_account  
206     LEFT JOIN bal_in        bi ON bi.account_rk = a.account_rk  
207     LEFT JOIN bal_out       bo ON bo.account_rk = a.account_rk  
208     LEFT JOIN turn_period tp ON tp.account_rk = a.account_rk  
209 )  
210 INSERT INTO dm.dm_f101_round_f(  
211     from_date, to_date, chapter, ledger_account, characteristic,  
212     balance_in_rub, balance_in_val, balance_in_total,  
213     turn_deb_rub, turn_deb_val, turn_deb_total,  
214     turn_cre_rub, turn_cre_val, turn_cre_total,  
215     balance_out_rub, balance_out_val, balance_out_total)  
216 SELECT v_from_date, v_to_date,  
217     MAX(chapter), ledger_account, MAX(char_type),  
218     SUM(CASE WHEN currency_code IN ('810','643') THEN bal_in_rub ELSE 0 END),  
219     SUM(CASE WHEN currency_code NOT IN ('810','643') THEN bal_in_rub ELSE 0 END),  
220     SUM(bal_in_rub),  
221     SUM(CASE WHEN currency_code IN ('810','643') THEN turn_deb_rub_total ELSE 0 END),  
222     SUM(CASE WHEN currency_code NOT IN ('810','643') THEN turn_deb_rub_total ELSE 0 END),  
223     SUM(turn_deb_rub_total),  
224     SUM(CASE WHEN currency_code IN ('810','643') THEN turn_cre_rub_total ELSE 0 END),  
225     SUM(CASE WHEN currency_code NOT IN ('810','643') THEN turn_cre_rub_total ELSE 0 END),  
226     SUM(turn_cre_rub_total),  
227     SUM(CASE WHEN currency_code IN ('810','643') THEN bal_out_rub ELSE 0 END),  
228     SUM(CASE WHEN currency_code NOT IN ('810','643') THEN bal_out_rub ELSE 0 END),  
229     SUM(bal_out_rub)  
230 FROM enriched  
231 GROUP BY ledger_account;  
232  
233 GET DIAGNOSTICS v_rows = ROW_COUNT;  
234  
235 INSERT INTO logs.etl_log(process, object_name, event, started_at, finished_at, rows_affected, extra)  
236 VALUES ('fill_f101_round_f', 'dm.dm_f101_round_f', 'END',  
237     v_started, clock_timestamp(), v_rows,  
238     format('period=%s..%s', v_from_date, v_to_date));  
239 EXCEPTION WHEN OTHERS THEN  
240     INSERT INTO logs.etl_log(process, object_name, event, started_at, finished_at, extra)  
241     VALUES ('fill_f101_round_f', 'dm.dm_f101_round_f', 'ERROR',  
242         v_started, clock_timestamp(), SQLERRM);  
243     RAISE;  
244 END;
```

Рис. 10. *fill_f101_round_f* — INSERT с разделением сумм по валютам (810/643 — рубли, всё прочее — валюта/драгметаллы), GET DIAGNOSTICS, EXCEPTION.

8. Чистый старт перед демонстрацией

Чтобы наглядно показать, что данные в DM-витрины приходят именно из наших процедур, перед запуском скриптов очищаем витрины и удаляем предыдущие лог-события от тех же процессов:

```
TRUNCATE TABLE dm.dm_account_turnover_f, dm.dm_account_balance_f;
```

```
DELETE FROM logs.etl_log
```

```
WHERE process IN ('fill_account_turnover_f','fill_account_balance_f','seed_balance_2017_12_31');
```

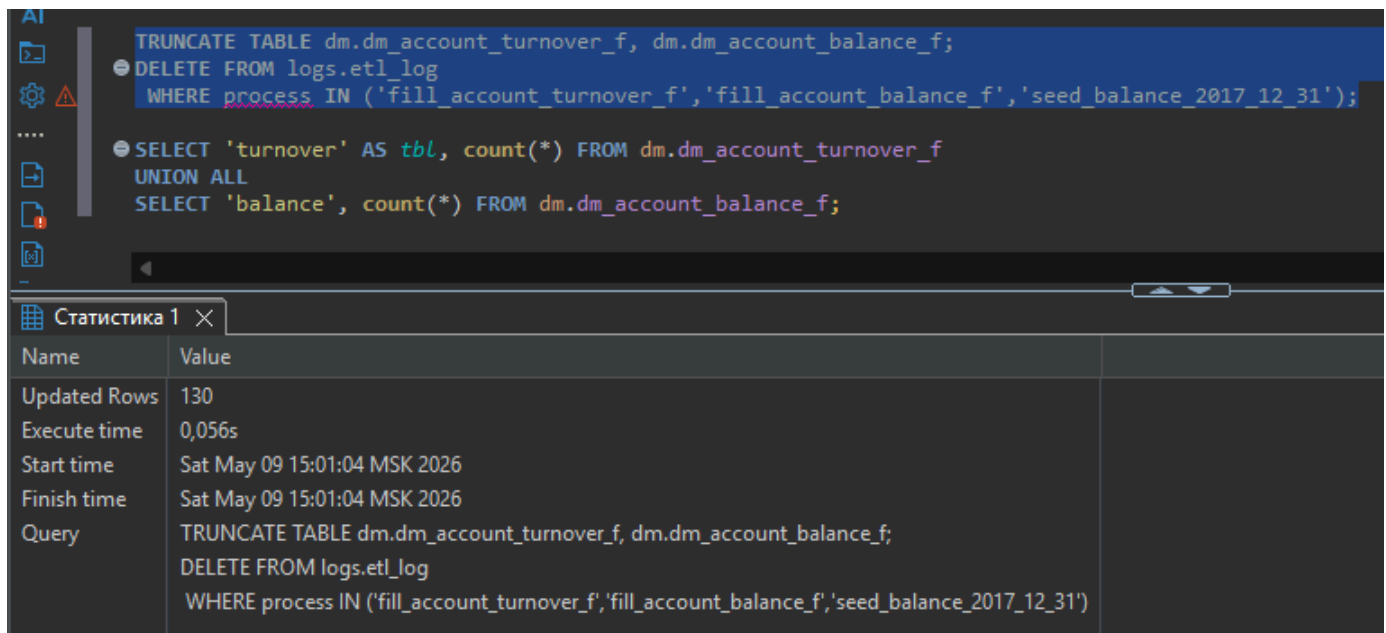


Рис. 11. Выполнение TRUNCATE + DELETE logs в DBeaver, Updated Rows: 130 за 0.056 с.

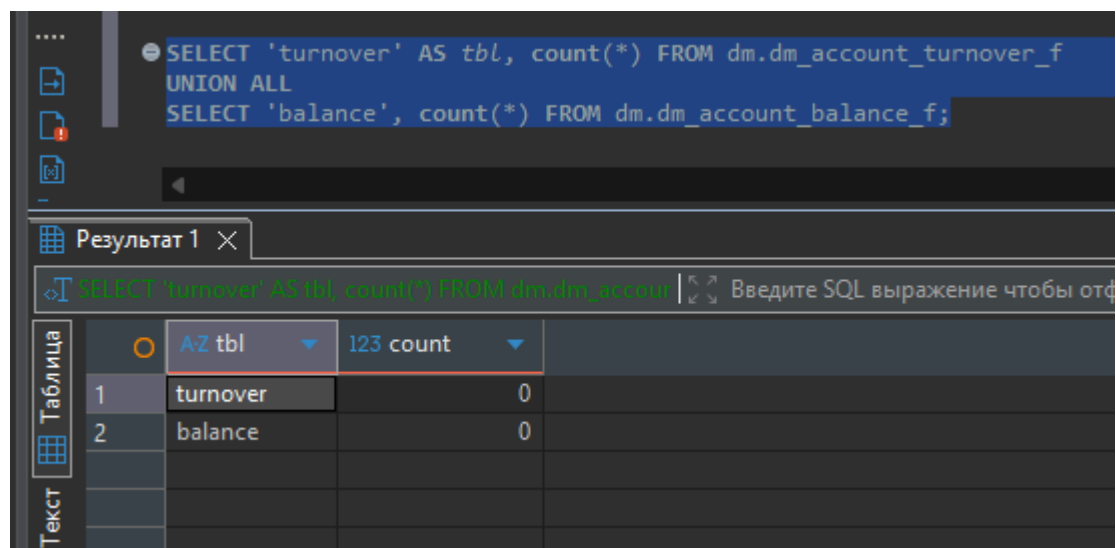


Рис. 12. Контрольный запрос — обе витрины ДМ пусты (0 строк).

9. Предварительное заполнение остатков на 31.12.2017

Процедура fill_account_balance_f(i_OnDate) опирается на CTE prev_bal — остатки за i_OnDate – 1 день. Чтобы расчёт за 01.01.2018 имел корректную «отправную точку», предварительно заполняем dm.dm_account_balance_f за 31.12.2017 данными из ds.ft_balance_f. Согласно ТЗ — поля on_date, account_rk, balance_out копируются один-в-один, а balance_out_rub считается как balance_out × курс валюты счёта на 31.12.2017 (если курса нет — ×1).

Реализован отдельным Python-скриптом seed_balance_2017_12_31.py. Скрипт сам пишет события START и END в logs.etl_log.

```

1  from __future__ import annotations
2
3  from datetime import datetime
4
5  import psycopg2
6
7  from db import DB_CONFIG
8
9  SQL_INIT = """
10 DELETE FROM dm.dm_account_balance_f WHERE on_date = DATE '2017-12-31';
11
12 INSERT INTO dm.dm_account_balance_f(on_date, account_rk, balance_out, balance_out_rub)
13 SELECT b.on_date,
14        b.account_rk,
15        b.balance_out,
16        b.balance_out * COALESCE(er.reduced_course, 1)
17 FROM ds.ft_balance_f b
18 LEFT JOIN ds.md_exchange_rate_d er
19     ON er.currency_rk = b.currency_rk
20     AND b.on_date BETWEEN er.data_actual_date
21        AND COALESCE(er.data_actual_end_date, DATE '9999-12-31')
22 WHERE b.on_date = DATE '2017-12-31';
23 """
24
25
26 def main() -> None:
27     started = datetime.now()
28     with psycopg2.connect(**DB_CONFIG) as conn, conn.cursor() as cur:
29         cur.execute(
30             "INSERT INTO logs.etl_log(process, object_name, event, started_at, extra) "
31             "VALUES (%s,%s,%s,%s,%s)",
32             ("seed_balance_2017_12_31", "dm.dm_account_balance_f", "START",
33              started, "init from ds.ft_balance_f"),
34         )
35         cur.execute(SQL_INIT)
36         cur.execute("SELECT count(*) FROM dm.dm_account_balance_f WHERE on_date = '2017-12-31';")
37         rows = cur.fetchone()[0]
38         cur.execute(
39             "INSERT INTO logs.etl_log(process, object_name, event, started_at, finished_at, rows_affected) "
40             "VALUES (%s,%s,%s,%s,%s,%s)",
41             ("seed_balance_2017_12_31", "dm.dm_account_balance_f", "END",
42              started, datetime.now(), rows),
43         )
44         print(f"[OK] заполнено {rows} строк остатков на 31.12.2017")
45
46
47 if __name__ == "__main__":
48     main()
49

```

Рис. 13. `seed_balance_2017_12_31.py` — SQL-блок `INSERT ... LEFT JOIN ds.md_exchange_rate_d с COALESCE(reduced_course, 1)`.

"""Заполнение dm.dm_account_balance_f за 31.12.2017 из ds.ft_balance_f.

Обязательный шаг между задачей 1.1 и расчётом остатков задачи 1.2:

`balance_out_rub = balance_out * курс на 31.12.2017 (если курса нет - 1)`.

"""

`from __future__ import annotations`

`from datetime import datetime`

`import psycopg2`

```
from db import DB_CONFIG
```

```
SQL_INIT = """
```

```
DELETE FROM dm.dm_account_balance_f WHERE on_date = DATE '2017-12-31';
```

```
INSERT INTO dm.dm_account_balance_f(on_date, account_rk, balance_out, balance_out_rub)
```

```
SELECT b.on_date,
```

```
b.account_rk,
```

```
b.balance_out,
```

```
b.balance_out * COALESCE(er.reduced_cource, 1)
```

```
FROM ds.ft_balance_f b
```

```
LEFT JOIN ds.md_exchange_rate_d er
```

```
ON er.currency_rk = b.currency_rk
```

```
AND b.on_date BETWEEN er.data_actual_date
```

```
AND COALESCE(er.data_actual_end_date, DATE '9999-12-31')
```

```
WHERE b.on_date = DATE '2017-12-31';
```

```
"""
```

```
def main() → None:
```

```
started = datetime.now()
```

```
with psycopg2.connect(**DB_CONFIG) as conn, conn.cursor() as cur:
```

```
cur.execute(
```

```
"INSERT INTO logs.etl_log(process, object_name, event, started_at, extra) "
```

```
"VALUES (%s,%s,%s,%s,%s)",
```

```
("seed_balance_2017_12_31", "dm.dm_account_balance_f", "START",
```

```
started, "init from ds.ft_balance_f"),
```

```
)
```

```
cur.execute(SQL_INIT)
```

```
cur.execute("SELECT count(*) FROM dm.dm_account_balance_f WHERE on_date = '2017-12-31';")
```

```
rows = cur.fetchone()[0]
```

```
cur.execute(
```

```
"INSERT INTO logs.etl_log(process, object_name, event, started_at, finished_at, rows_affected) "
```

```

"VALUES (%s,%s,%s,%s,%s,%s)",
("seed_balance_2017_12_31", "dm.dm_account_balance_f", "END",
started, datetime.now(), rows),
)
print(f"[OK] заполнено {rows} строк остатков на 31.12.2017")
if __name__ == "__main__":
    main()

```

Запуск из терминала:

```

PS C:\Users\Maksim\Desktop\neo\hw_coursework_task1\python> python seed_balance_2017_12_31.py
[OK] заполнено 114 строк остатков на 31.12.2017
PS C:\Users\Maksim\Desktop\neo\hw_coursework_task1\python> |

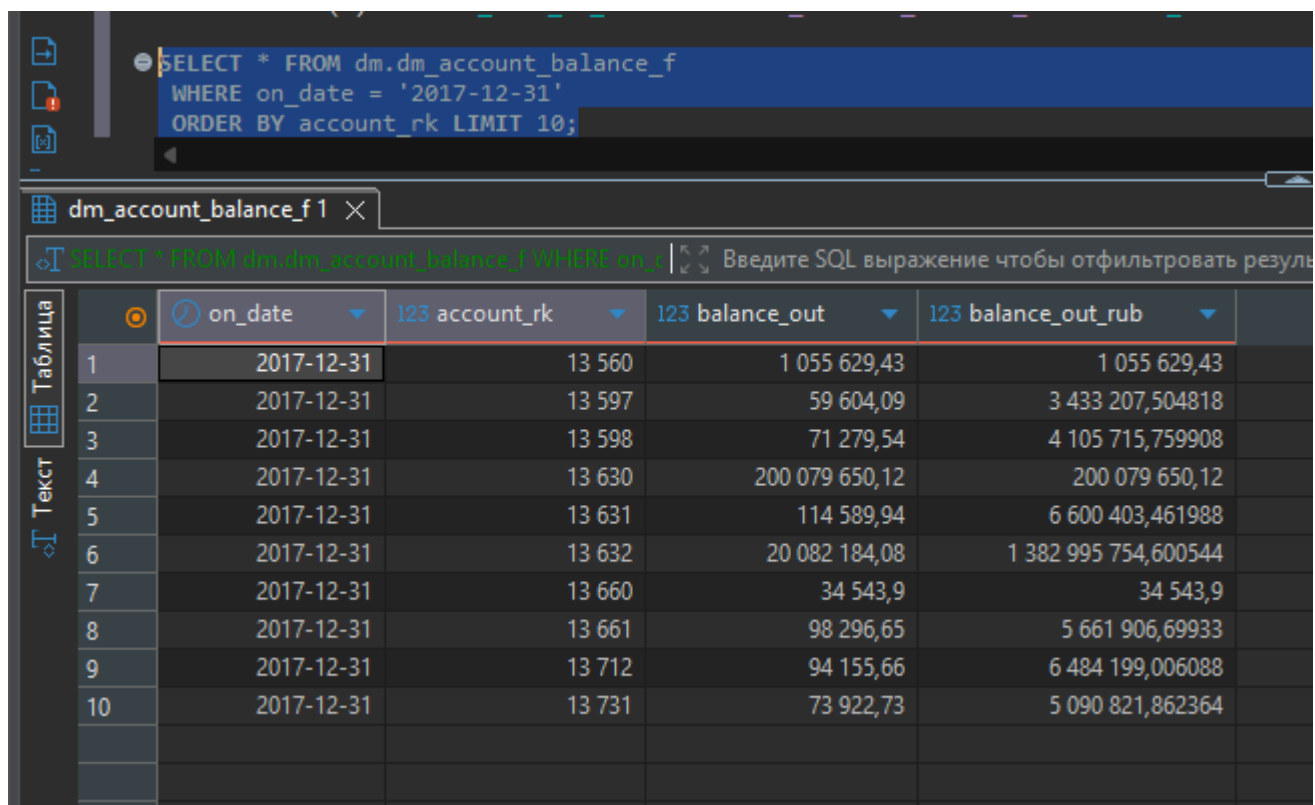
```

Рис. 14. `python seed_balance_2017_12_31.py` — заполнено 114 строк остатков на 31.12.2017.

The screenshot shows a SQL IDE interface. The top pane contains two SQL queries. The first query is highlighted in blue: `SELECT count(*) AS rows_2017_12_31 FROM dm.dm_account_balance_f WHERE on_date = '2017-12-31';`. The second query is `SELECT * FROM dm.dm_account_balance_f WHERE on_date = '2017-12-31' ORDER BY account_rk LIMIT 10;`. The bottom pane, titled 'Результат 1', shows the result of the first query. It includes a search bar with the text 'Введите SQL выражение чтобы отфильтровать результаты' and a table with one row and one column.

Таблица	123 rows_2017_12_31
1	114

Рис. 15. `SELECT count(*) FROM dm.dm_account_balance_f WHERE on_date = '2017-12-31' = 114` (все счета из `ds.ft_balance_f`).



SQL Query:

```
SELECT * FROM dm.dm_account_balance_f
WHERE on_date = '2017-12-31'
ORDER BY account_rk LIMIT 10;
```

Table View: dm_account_balance_f 1

SQL Expression: SELECT * FROM dm.dm_account_balance_f WHERE on_date = '2017-12-31' ORDER BY account_rk LIMIT 10

	on_date	123 account_rk	123 balance_out	123 balance_out_rub
1	2017-12-31	13 560	1 055 629,43	1 055 629,43
2	2017-12-31	13 597	59 604,09	3 433 207,504818
3	2017-12-31	13 598	71 279,54	4 105 715,759908
4	2017-12-31	13 630	200 079 650,12	200 079 650,12
5	2017-12-31	13 631	114 589,94	6 600 403,461988
6	2017-12-31	13 632	20 082 184,08	1 382 995 754,600544
7	2017-12-31	13 660	34 543,9	34 543,9
8	2017-12-31	13 661	98 296,65	5 661 906,69933
9	2017-12-31	13 712	94 155,66	6 484 199,006088
10	2017-12-31	13 731	73 922,73	5 090 821,862364

Рис. 16. Первые 10 строк остатков на 31.12.2017 — видны *balance_out* (валюта счёта) и *balance_out_rub* (рубли).

10. Расчёт оборотов и остатков за январь 2018

Скрипт `run_january.py` итерирует по датам с 01.01.2018 по 31.01.2018 и для каждой даты последовательно вызывает `CALL ds.fill_account_turnover_f(d)` и `CALL ds.fill_account_balance_f(d)`. Коммит делаем после каждого дня — это даёт реальные `started_at/finished_at` в логах (а не одно общее время транзакции) и устойчивость к ошибкам на середине месяца.

```

1  from __future__ import annotations
2
3  from datetime import date, datetime, timedelta
4
5  import psycopg2
6
7  from db import DB_CONFIG
8
9
10 def main() -> None:
11     start = date(2018, 1, 1)
12     end   = date(2018, 1, 31)
13
14     t0 = datetime.now()
15     with psycopg2.connect(**DB_CONFIG) as conn:
16         d = start
17         while d <= end:
18             with conn.cursor() as cur:
19                 cur.execute("CALL ds.fill_account_turnover_f(%s);", (d,))
20                 cur.execute("CALL ds.fill_account_balance_f(%s);", (d,))
21             conn.commit()
22             print(f"[OK] {d}")
23             d += timedelta(days=1)
24     print(f"[DONE] Январь 2018 рассчитан за {(datetime.now()-t0).total_seconds():.2f} с.")
25
26
27 if __name__ == "__main__":
28     main()
29

```

Рис. 17. run_january.py — цикл по дням января с CALL обеих процедур и conn.commit() после каждой пары.

"""Прогон витрин оборотов и остатков за каждый день января 2018.

Коммитим после каждого дня (а не один раз в конце), чтобы:

1. в логах logs.etl_log для каждой даты были реальные started_at/finished_at;
2. при сбое промежуточные дни оставались зафиксированы.

"""

from __future__ import annotations

from datetime import date, datetime, timedelta

import psycopg2

from db import DB_CONFIG

def main() → None:

start = date(2018, 1, 1)

end = date(2018, 1, 31)

```
t0 = datetime.now()

with psycopg2.connect(**DB_CONFIG) as conn:

    d = start

    while d <= end:

        with conn.cursor() as cur:

            cur.execute("CALL ds.fill_account_turnover_f(%s);", (d,))

            cur.execute("CALL ds.fill_account_balance_f(%s);", (d,))

        conn.commit()

        print(f"[OK] {d}")

        d += timedelta(days=1)

    print(f"[DONE] Январь 2018 рассчитан за {(datetime.now()-t0).total_seconds():.2f} с.")

if __name__ == "__main__":

    main()
```

Запуск из терминала:


```
[OK] 2018-01-04
[OK] 2018-01-05
[OK] 2018-01-06
[OK] 2018-01-07
[OK] 2018-01-08
[OK] 2018-01-09
[OK] 2018-01-10
[OK] 2018-01-11
[OK] 2018-01-12
[OK] 2018-01-13
[OK] 2018-01-14
[OK] 2018-01-15
[OK] 2018-01-16
[OK] 2018-01-17
[OK] 2018-01-18
[OK] 2018-01-19
[OK] 2018-01-20
[OK] 2018-01-21
[OK] 2018-01-22
[OK] 2018-01-23
[OK] 2018-01-24
[OK] 2018-01-25
[OK] 2018-01-26
[OK] 2018-01-27
[OK] 2018-01-28
[OK] 2018-01-29
[OK] 2018-01-30
[OK] 2018-01-31
[DONE] Январь 2018 рассчитан за 0.83 с.
PS C:\Users\Maksim\Desktop\neo\hw_coursework_task1\python> |
```

Рис. 18. `python run_january.py` — все 31 день обработаны, `[DONE]` Январь 2018 рассчитан за 0.83 с.

11. Результаты расчёта в витринах DM

Сводка по витрине оборотов: видно, что строки появляются только в датах с проводками. В исходном датасете проводки присутствуют не за все 31 день, а только за рабочие дни — поэтому в витрине 17 уникальных дат с 09.01 по 31.01. Это полностью соответствует требованию ТЗ «если по счету не было проводок в дату расчёту, то он не должен попадать в витрину».

```

SELECT on_date,
       count(*) AS accounts,
       round(sum(credit_amount_rub)::numeric,2) AS sum_cre_rub,
       round(sum(debet_amount_rub)::numeric,2) AS sum_deb_rub
FROM dm.dm_account_turnover_f
GROUP BY on_date
ORDER BY on_date;

```

on_date	123 accounts	123 sum_cre_rub	123 sum_deb_rub
2018-01-09	212	463 355 876,32	407 814 211,67
2018-01-10	175	260 495 758,69	158 182 937,51
2018-01-11	183	299 823 743,35	234 006 442,89
2018-01-12	182	221 286 599,04	155 924 283,12
2018-01-15	164	280 127 699,96	179 136 286,5
2018-01-16	196	267 648 537,33	241 473 433,57
2018-01-17	157	176 874 562,94	202 132 479,09
2018-01-18	161	311 573 674,26	242 960 369,01
2018-01-19	159	208 246 350,24	157 487 339,39
2018-01-22	167	270 920 101,87	209 640 221,54
2018-01-23	165	225 036 179,18	155 257 669,93
2018-01-24	164	145 399 219,41	131 014 190,2
2018-01-25	184	230 321 340,88	218 888 356,7
2018-01-26	165	181 174 790,25	174 857 857,58
2018-01-29	171	314 913 013,1	242 650 016,84
2018-01-30	189	194 481 083,46	199 375 034,95
2018-01-31	188	217 594 073,75	201 033 686,7

Рис. 19. *dm.dm_account_turnover_f* — обороты по дням января (17 дат с проводками), кол-во счетов 157–212, суммы в рублях.

Сводка по витрине остатков: 32 уникальных даты — 31.12.2017 (seed, 114 счетов) + январь (по 112 активных счетов в день по *md_account_d*). Видна динамика суммарного остатка: с 3,326 млрд руб. на 31.12.2017 до 2,534 млрд руб. на 27.01.2018 — это естественное снижение по мере накопления оборотов и зависит от соотношения А/П-счетов в датасете. Важно: в днях без оборотов остатки всё равно записываются (счёт «протягивается» с предыдущего дня).

Пример топ-10 строк витрины оборотов за 09.01.2018, отсортированный по `credit_amount_rub`. Видно, что `credit_amount` (валюта счёта) и `credit_amount_rub` различаются для счетов в иностранной валюте — там применён курс из `ds.md_exchange_rate_d`:

```

SELECT on_date, account_rk,
       credit_amount, credit_amount_rub,
       debit_amount, debit_amount_rub
FROM dm.dm_account_turnover_f
WHERE on_date = '2018-01-09'
ORDER BY credit_amount_rub DESC NULLS LAST
LIMIT 10;

```

	on_date	123 account_rk	123 credit_amount	123 credit_amount_rub	123 debit_amount	123 debit_amount_rub
1	2018-01-09	13 632	3 262 575,86	224 683 159,235448	2 262 297,1	155 797 161,92628
2	2018-01-09	13 630	80 020 220,76	80 020 220,76	46 566 724,02	46 566 724,02
3	2018-01-09	13 631	909 738,98	52 401 147,195796	1 248 433,78	71 910 035,414756
4	2018-01-09	34 157 147	274 922,35	15 835 582,34447	20 093,24	1 157 374,642648
5	2018-01-09	14 138	224 344,95	15 449 918,80266	347 114,78	23 904 684,131304
6	2018-01-09	10 006 159	8 603 401,79	8 603 401,79	2 548 691,02	2 548 691,02
7	2018-01-09	409 685 020	7 513 660,51	7 513 660,51	18 588 118,84	18 588 118,84
8	2018-01-09	1 761 691	7 495 122,1	7 495 122,1	351 664,5	351 664,5
9	2018-01-09	393 808 409	7 333 400,2	7 333 400,2	0	0
10	2018-01-09	13 906	113 539,46	6 695 910,175878	19 003,47	1 120 716,340821

Рис. 21. `dm.dm_account_turnover_f` за 09.01.2018, топ-10 — `credit_amount`, `credit_amount_rub`, `debit_amount`, `debit_amount_rub`.

12. Логирование процедур

Для логирования используется таблица `logs.etl_log` из задачи 1.1 (`process`, `object_name`, `event`, `started_at`, `finished_at`, `rows_affected`, `extra`). На каждый вызов процедуры в логе появляются ровно две записи — событие `START` (без `finished_at`) и событие `END` (с `finished_at` и количеством вставленных строк). В `extra` `END`-записи дополнительно сохраняется `parent_log_id` (ссылка на `START` той же пары) и `duration_sec` — фактическая длительность.

Решение хранить всё в единой таблице `logs.etl_log` для всех ETL-процессов курсовой выбрано по двум причинам: (1) единое место для трассировки — поднимая один и тот же запрос, можно увидеть и загрузку CSV в DS, и расчёт оборотов, и расчёт остатков, и форму 101, не дёргая разные таблицы; (2) расширяемость — для нового процесса достаточно завести новое значение колонки `process`. Поле `extra` оставлено как свободный текст, чтобы туда писать произвольные параметры (дату расчёта, имя файла, `parent_log_id`, `duration_sec`, текст ошибки).

```

SELECT log_id, process, event,
       to_char(started_at, 'HH24:MI:SS.MS') AS started,
       to_char(finished_at, 'HH24:MI:SS.MS') AS finished,
       rows_affected, extra
FROM logs.etl_log
WHERE process IN ('fill_account_turnover_f', 'fill_account_balance_f', 'seed_balance_2017_12_31')
ORDER BY log_id
LIMIT 16;

```

	log_id	process	event	started	finished	rows_affected	extra
1	271	seed_balance_2017_12_31	START	15:02:04.591	[NULL]	[NULL]	init from ds.ft_balance_f
2	272	seed_balance_2017_12_31	END	15:02:04.591	15:02:04.763	114	[NULL]
3	273	fill_account_turnover_f	START	12:03:25.137	[NULL]	[NULL]	on_date=2018-01-01
4	274	fill_account_turnover_f	END	12:03:25.137	12:03:25.182	0	on_date=2018-01-01; parent_log_id=273; duration_sec=0.044
5	275	fill_account_balance_f	START	12:03:25.190	[NULL]	[NULL]	on_date=2018-01-01
6	276	fill_account_balance_f	END	12:03:25.190	12:03:25.207	112	on_date=2018-01-01; parent_log_id=275; duration_sec=0.017
7	277	fill_account_turnover_f	START	12:03:25.214	[NULL]	[NULL]	on_date=2018-01-02
8	278	fill_account_turnover_f	END	12:03:25.214	12:03:25.224	0	on_date=2018-01-02; parent_log_id=277; duration_sec=0.010
9	279	fill_account_balance_f	START	12:03:25.226	[NULL]	[NULL]	on_date=2018-01-02
10	280	fill_account_balance_f	END	12:03:25.226	12:03:25.228	112	on_date=2018-01-02; parent_log_id=279; duration_sec=0.002
11	281	fill_account_turnover_f	START	12:03:25.234	[NULL]	[NULL]	on_date=2018-01-03
12	282	fill_account_turnover_f	END	12:03:25.234	12:03:25.246	0	on_date=2018-01-03; parent_log_id=281; duration_sec=0.012
13	283	fill_account_balance_f	START	12:03:25.247	[NULL]	[NULL]	on_date=2018-01-03
14	284	fill_account_balance_f	END	12:03:25.247	12:03:25.250	112	on_date=2018-01-03; parent_log_id=283; duration_sec=0.003
15	285	fill_account_turnover_f	START	12:03:25.256	[NULL]	[NULL]	on_date=2018-01-04
16	286	fill_account_turnover_f	END	12:03:25.256	12:03:25.266	0	on_date=2018-01-04; parent_log_id=285; duration_sec=0.010

Рис. 22. logs.etl_log — первые 16 событий: пары START/END для seed_balance_2017_12_31 и нескольких дней января. В extra END-записей видны parent_log_id и duration_sec.

Сводка по логу — количество START и END для каждого процесса. Числа в идеале должны попарно совпадать; у нас по 31 паре START/END у каждой из двух процедур и 1 пара у seed-скрипта:

```

SELECT process, event, count(*) AS n
FROM logs.etl_log
WHERE process IN ('fill_account_turnover_f', 'fill_account_balance_f', 'seed_balance_2017_12_31')
GROUP BY process, event
ORDER BY process, event;

```

	process	event	n
1	fill_account_balance_f	END	31
2	fill_account_balance_f	START	31
3	fill_account_turnover_f	END	31
4	fill_account_turnover_f	START	31
5	seed_balance_2017_12_31	END	1
6	seed_balance_2017_12_31	START	1

Рис. 23. Сводка по logs.etl_log: fill_account_balance_f — 31 START + 31 END, fill_account_turnover_f — 31 START + 31 END, seed_balance_2017_12_31 — 1 START + 1 END.

13. Демонстрация идемпотентности повторного вызова

ТЗ требует возможности перезапускать расчёт за одну и ту же дату многократно — поэтому в начале каждой процедуры стоит DELETE по on_date. Демонстрация: считаем число строк за 15.01.2018 до повторного вызова, выполняем CALL обеих процедур ещё раз, считаем строки после.

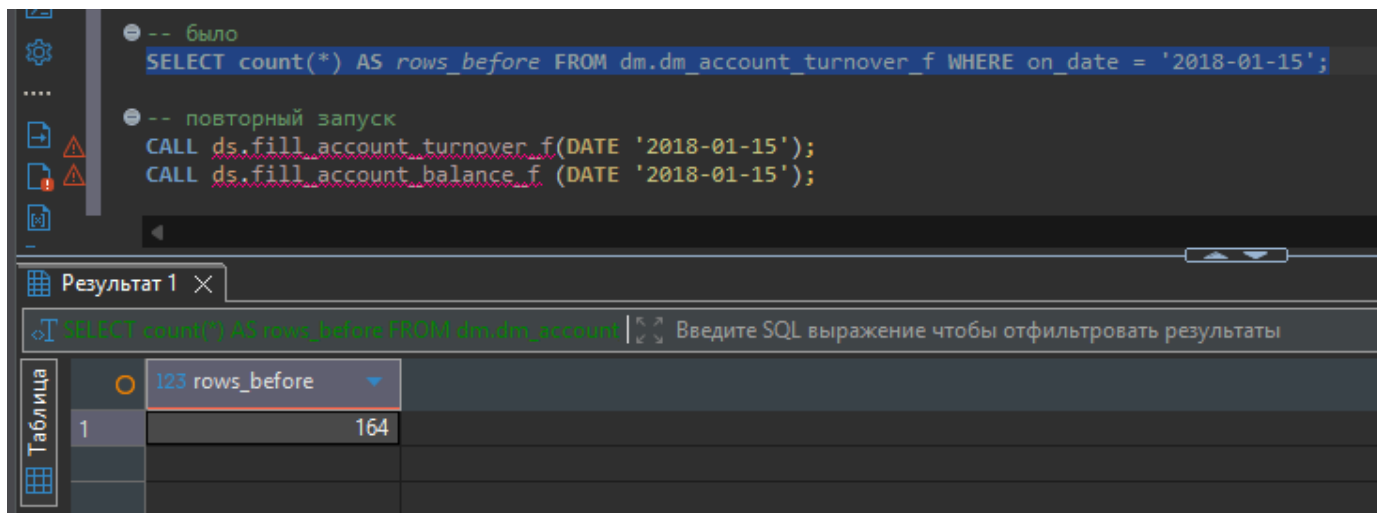


Рис. 24. До повторного запуска: `dm.dm_account_turnover_f` за 2018-01-15 содержит 164 строки.

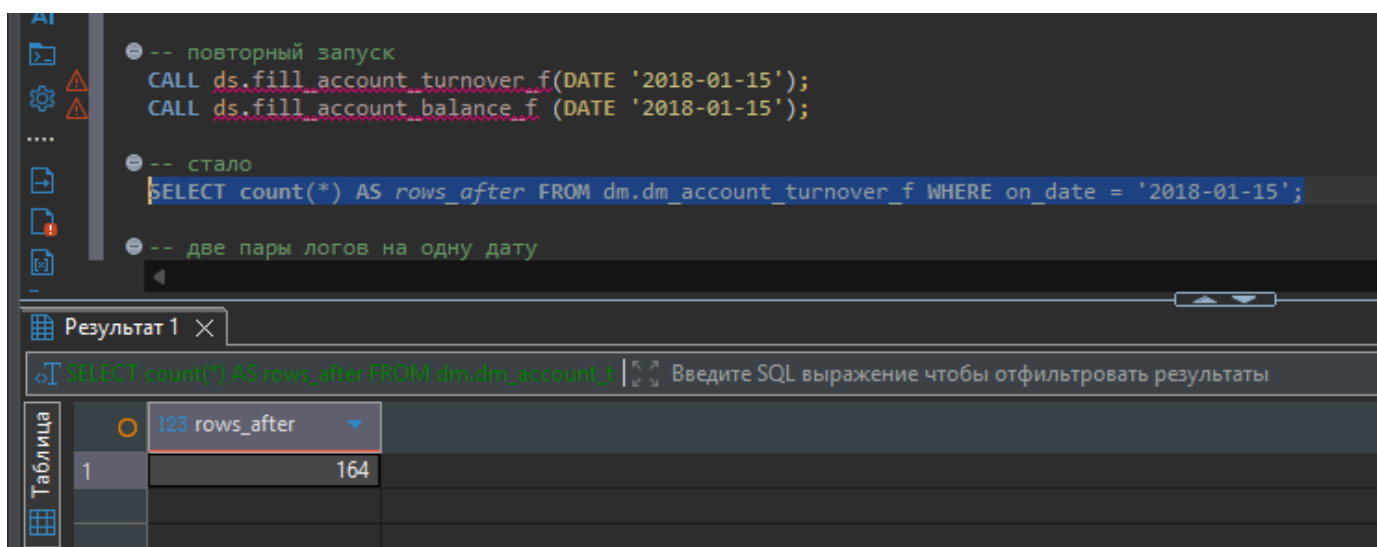


Рис. 25. После повторного `CALL` — снова 164 строки. Дублирования нет — значит `DELETE` + `INSERT` отработывают корректно.

А вот в логе появляется новая пара `START/END` для каждой процедуры — то есть видно ровно два независимых запуска одной и той же даты:

SQL Query:

```
-- две пары логов на одну дату
SELECT log_id, process, event, to_char(started_at, 'HH24:MI:SS.MS') AS started, rows_affected
FROM logs.etl_log
WHERE process IN ('fill_account_turnover_f', 'fill_account_balance_f')
AND extra LIKE '%2018-01-15%'
ORDER BY log_id;
```

Table: etl_log 1

SQL Filter: `SELECT log_id, process, event, to_char(started_at, 'HH24:MI:SS.MS') AS started, rows_affected`

	log_id	process	event	started	rows_affected
1	329	fill_account_turnover_f	START	12:03:25.462	[NULL]
2	330	fill_account_turnover_f	END	12:03:25.462	164
3	331	fill_account_balance_f	START	12:03:25.474	[NULL]
4	332	fill_account_balance_f	END	12:03:25.474	112
5	397	fill_account_turnover_f	START	15:06:20.524	[NULL]
6	398	fill_account_turnover_f	END	15:06:20.524	164
7	399	fill_account_balance_f	START	15:06:25.004	[NULL]
8	400	fill_account_balance_f	END	15:06:25.004	112

Рис. 26. logs.etl_log по записям с extra LIKE '%2018-01-15%' — две пары START/END у обеих процедур (log_id 329–332 первый запуск, 397–400 повторный).

14. Итог по задаче 1.2

- созданы и применены к БД процедуры ds.fill_account_turnover_f и ds.fill_account_balance_f, точно соответствующие ТЗ;
- предусмотрена идемпотентность — DELETE WHERE on_date = i_OnDate в начале каждой процедуры;
- реализован seed-скрипт для предварительного заполнения остатков на 31.12.2017 из ds.ft_balance_f, с пересчётом в рубли через ds.md_exchange_rate_d;
- обе витрины рассчитаны за каждый день января 2018 — суммарно 17 дат с оборотами и 32 даты с остатками (включая seed);
- логирование START/END/ERROR ведётся в единую таблицу logs.etl_log из 1.1 — с parent_log_id и duration_sec в extra;
- идемпотентность подтверждена реальным демо: при повторном вызове за 15.01.2018 число строк не изменилось (164 → 164), а в логе появилась новая пара START/END.

Этим закрываются все требования задачи 1.2 курсовой работы.

15. Ссылки

Репозиторий GitHub (ветка 1_2): https://github.com/kodi842/neoflex_coursework/tree/1_2

Видео-демонстрация (Google Drive, все 4 видео по курсовой): <https://drive.google.com/drive/folders/1ZMhXnrIE3nR8zUqTHC2uECeeShwDaIFG?usp=sharing>